

Privacy-Preserving Source Code Vulnerability Detection & Repair

Retrieval-Augmented LLMs for Visual Studio Code

PRESENTED BY

Md Hafizur Rahman

Mat. No. 810641 · M.Sc. Web Engineering

SUPERVISOR

Abubaker Gaber (Internal)

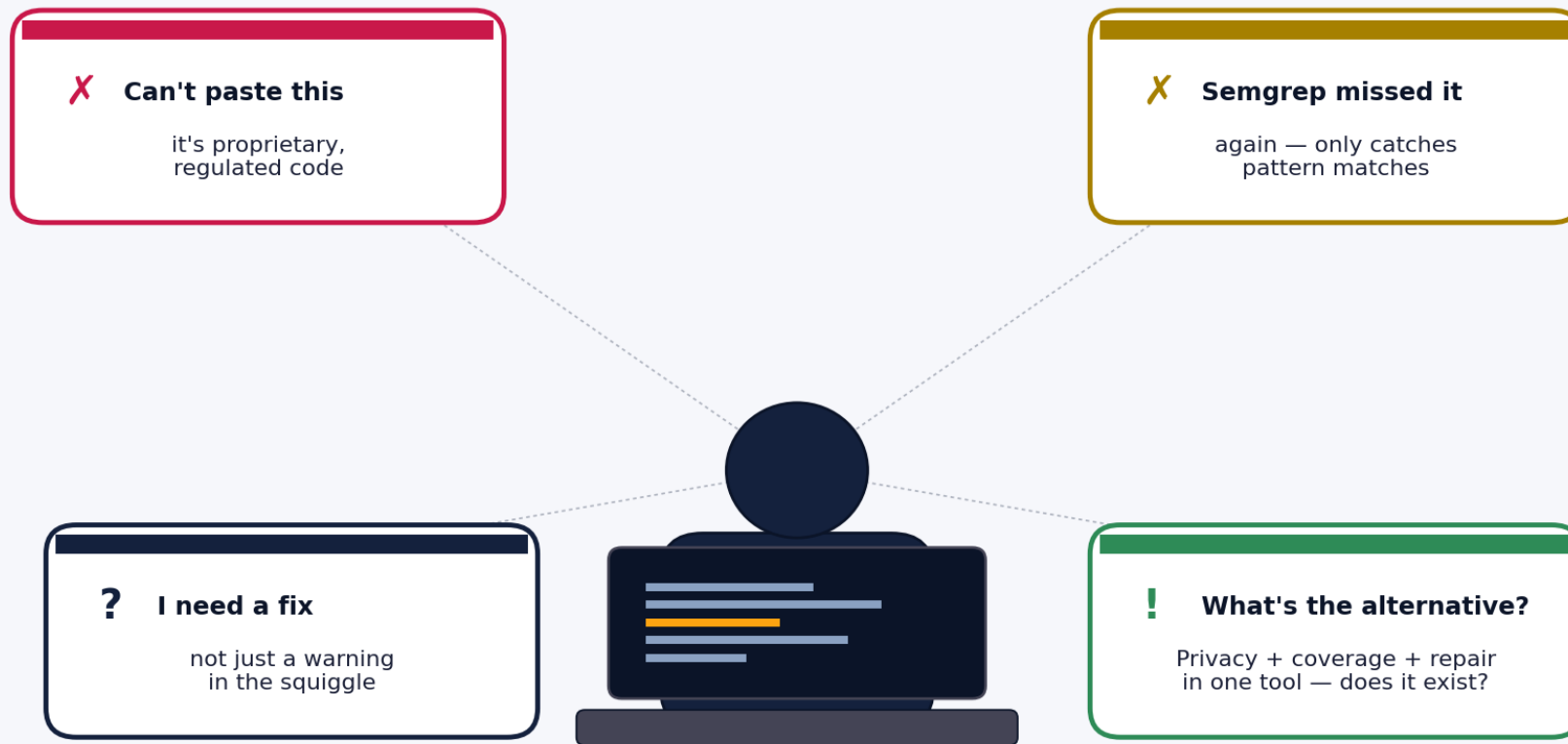
EXAMINER

Dr.-Ing. Sebastian Heil

DATE

June 16, 2026

The Developer's Dilemma



Privacy, coverage, and repair quality — pick at most two.

Today You Pick: Send Code Away — or Get No Help

❌ Cloud LLM (Copilot, ChatGPT, Cursor)

Source code crosses the network on every request.

Blocked outright in finance, healthcare, defence,
and air-gapped environments.

Strong reasoning • zero privacy guarantees.

❌ Local SAST (Semgrep, CodeQL, ESLint)

Stays local. Cannot reason across files or generate fixes.

Semgrep canonical recall on our JS/TS corpus: 12.68 %

Strong privacy • poor coverage • no repair.

Either choice forces a trade-off — privacy or capability, never both.

No Tool Combines All Three



Local Inference

Code never leaves the developer's machine.
Verifiable network boundary.



Retrieval-Augmented

Grounded in CWE / OWASP knowledge.
Not just pattern matching.



Developer-Controlled Repair

Structured fix returned.
Developer applies via Quick Fix.
No silent rewrites.

This thesis builds and evaluates a tool that delivers all three.

Research Questions and Requirements

RESEARCH QUESTIONS

- RQ1** Can a local LLM on consumer hardware reach acceptable detection quality under a strict privacy envelope?
- RQ2** Does grounding the LLM in retrieved CWE / OWASP knowledge (RAG) improve detection?
- RQ3** Is in-IDE latency acceptable for developer workflows?

FIVE REQUIREMENTS

R1 Accuracy

precision, recall, F1

R2 Consistency

JSON parse, inter-run

R3 Repair Quality

manual + auto-applicable

R4 Usability

latency, IDE integration

R5 Privacy

zero exfiltration, signed corpus

R1-R5 use fixed four-level threshold scales set in Chapter 2 — before evaluation.

Three Existing Approaches

CLOUD LLM ASSISTANTS

GitHub Copilot

Microsoft · 2021

ChatGPT / GPT-4

OpenAI · 2022

Cursor

Anysphere · 2023

Claude Code

Anthropic · 2024

LOCAL SAST

Semgrep

Rule-based · OSS

CodeQL

GitHub · taint analysis

ESLint security plugins

Lint-level

Bandit / SpotBugs

Other languages

ACADEMIC LLM-FOR-SEC

IRIS + GPT-4

Li et al. · ICLR 2025

SecRepair

Islam et al. · NDSS 2024

LLMSecGuard

Kavian et al. · EASE 2024

RESCUE

Shi & Zhang · 2025

No Tool Satisfies R1 + R3 + R5 Together

Solution / Approach	R1 ACCURACY	R2 CONSISTENCY	R3 REPAIR	R4 USABILITY	R5 PRIVACY
GitHub Copilot	✓	●	✓	✓	✗
Semgrep	●	✓	✗	✓	✓
CodeQL	●	✓	✗	●	✓
IRIS (GPT-4)	●	●	✗	✗	✗
SecRepair	●	●	●	✗	●

LEGEND

- ✓ High / Satisfied
- Partial
- ✗ Not satisfied

THE GAP

No existing tool delivers privacy + repair + IDE-grade latency together.

One Pipeline, Four Developer Workflows



Real-time Diagnostics

Inline squiggles per function as you save.



On-demand File Scan

Right-click → scan the current file.



Interactive Q&A

Ask about a finding in a side panel.



Workspace Audit

Batch scan the whole project.

ALL FOUR WORKFLOWS HIT THE SAME PIPELINE

INPUT



STAGE 1
Detection (LLM + RAG)



Consensus
filter



STAGE 2
Repair (LLM)



QUICK
FIX

Two-Stage Local Pipeline



Stage 1 — Detection

- JSON-mode LLM call · schema-pinned output
- Optional RAG context · top-k CWE / OWASP snippets
- Three deterministic runs · seed = 42, temperature = 0
- Inter-run confidence gate · ≥ 2 -of-3 agreement (0.67)
- Returns: category, severity, line range, evidence

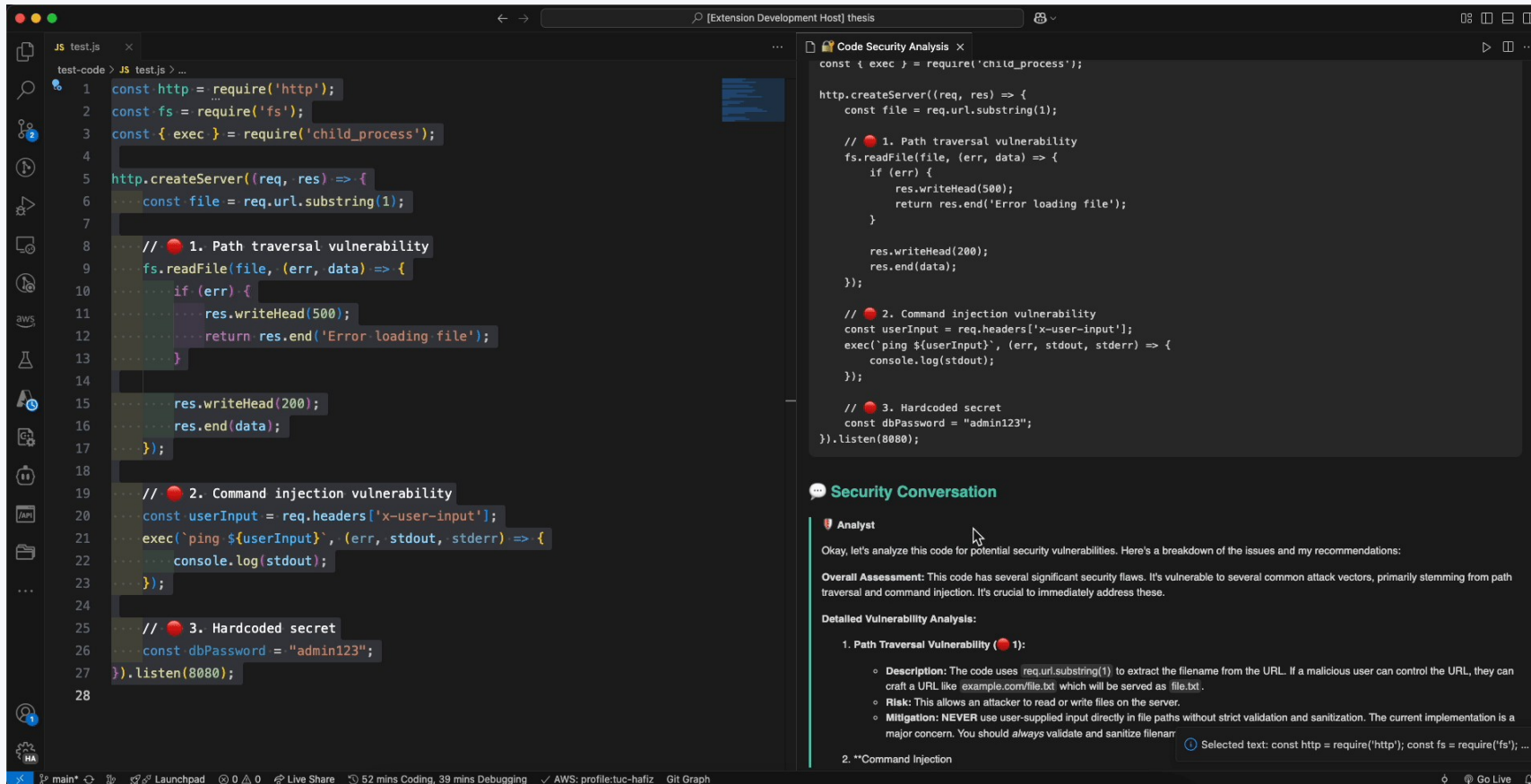


Stage 2 — Repair

- Separate structured call · invoked only when detected
- Returns { code, language } · 18 explicit abstentions allowed
- Validator parses output with @babel/parser · rejects prose
- Surfaced as a VS Code Quick Fix
- Developer applies the patch · no silent rewrites

Privacy by construction · Ollama bound to loopback · embeddings local · no telemetry

Live in VS Code



```
1 const http = require('http');
2 const fs = require('fs');
3 const { exec } = require('child_process');
4
5 http.createServer((req, res) => {
6   const file = req.url.substring(1);
7
8   // 1. Path traversal vulnerability
9   fs.readFile(file, (err, data) => {
10     if (err) {
11       res.writeHead(500);
12       return res.end('Error loading file');
13     }
14
15     res.writeHead(200);
16     res.end(data);
17   });
18
19   // 2. Command injection vulnerability
20   const userInput = req.headers['x-user-input'];
21   exec(`ping ${userInput}`, (err, stdout, stderr) => {
22     console.log(stdout);
23   });
24
25   // 3. Hardcoded secret
26   const dbPassword = "admin123";
27   }).listen(8080);
28
```

Code Security Analysis

```
const { exec } = require('child_process');

http.createServer((req, res) => {
  const file = req.url.substring(1);

  // 1. Path traversal vulnerability
  fs.readFile(file, (err, data) => {
    if (err) {
      res.writeHead(500);
      return res.end('Error loading file');
    }

    res.writeHead(200);
    res.end(data);
  });

  // 2. Command injection vulnerability
  const userInput = req.headers['x-user-input'];
  exec(`ping ${userInput}`, (err, stdout, stderr) => {
    console.log(stdout);
  });

  // 3. Hardcoded secret
  const dbPassword = "admin123";
}).listen(8080);
```

Security Conversation

Analyst

Okay, let's analyze this code for potential security vulnerabilities. Here's a breakdown of the issues and my recommendations:

Overall Assessment: This code has several significant security flaws. It's vulnerable to several common attack vectors, primarily stemming from path traversal and command injection. It's crucial to immediately address these.

Detailed Vulnerability Analysis:

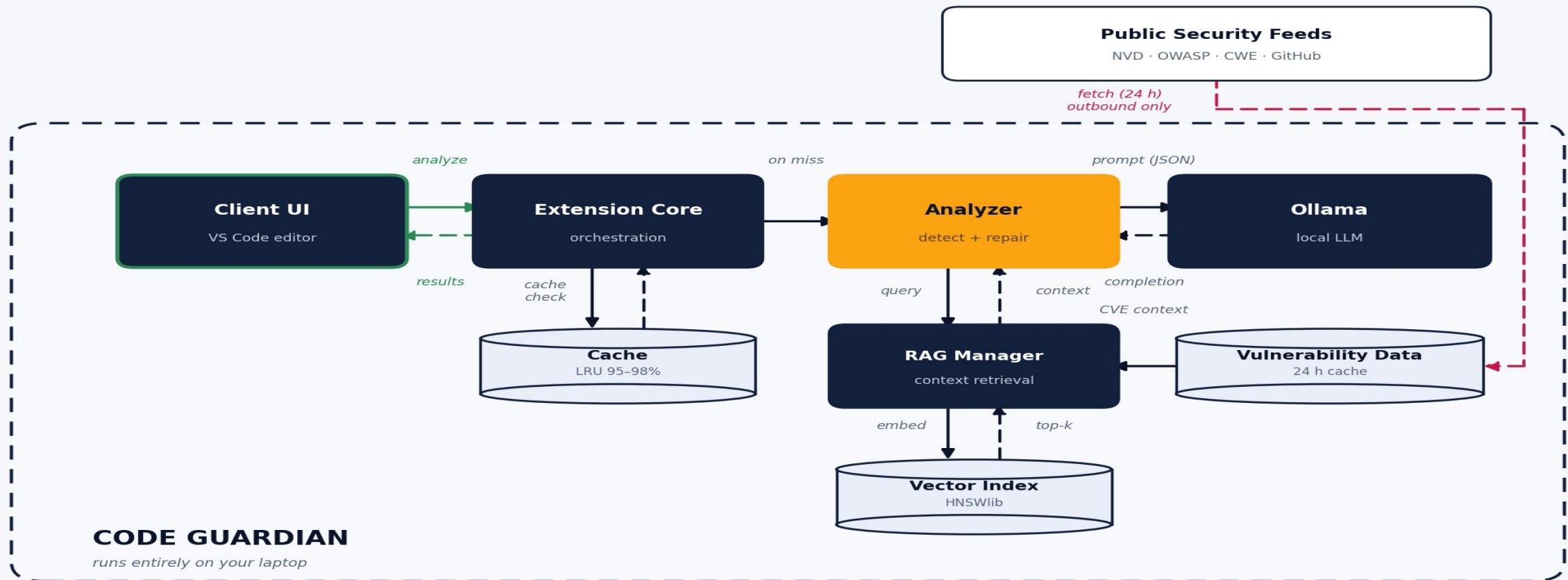
- 1. Path Traversal Vulnerability (1):**
 - Description:** The code uses `req.url.substring(1)` to extract the filename from the URL. If a malicious user can control the URL, they can craft a URL like `example.com/file.txt` which will be served as `file.txt`.
 - Risk:** This allows an attacker to read or write files on the server.
 - Mitigation:** NEVER use user-supplied input directly in file paths without strict validation and sanitization. The current implementation is a major concern. You should always validate and sanitize filename.
- 2. Command Injection**



SCAN · Install from VS Code Marketplace

Editor with flagged findings (path traversal · command injection · hardcoded secret) and the Code Security Analysis panel.

System Architecture



Only public vulnerability look-ups (red) ever leave the machine — your code never does.

Privacy by Construction



Loopback-only Ollama

Bound to 127.0.0.1. Network isolation arm of the threat model — empirically verified.



Signed RAG Corpus

Ed25519 manifest verified at load. Provenance arm of the threat model — corpus cannot be silently rewritten.



Pinned Container

Node 20.19.0-alpine · npm ci · seed = 42 · byte-identical runs.



Prompt-Injection Harness

12 cases of crafted prompts attempting exfiltration. leakFreeRate = 100 %.

Evaluation Setup



Corpus

Curated JS / TS · 101 cases
(71 vulnerable + 30 secure)
14 CWE categories

External: 15 cases from NodeGoat,
Juice Shop, 3 named CVEs

Whole-project scan: OWASP NodeGoat



Configurations

5 Ollama models × {LLM-only, LLM+RAG}
= 10 configurations

+ 3 SAST baselines
(Semgrep, CodeQL, ESLint)

3 runs per sample · deterministic
→ 303 invocations per config



Statistical Discipline

Held-out 71 / 30 test split
(threshold tuning off the test set)

McNemar with Bonferroni
across paired model comparisons

Exact-binomial CIs for small-n rates

Headline Numbers

R1 — DETECTION

71.43 %

F1 · qwen3:8b + RAG

precision 72.46 % · recall 70.42 %

Gated $\geq 0.67 \rightarrow$ 74.07 % F1, 78.13 % precision

R3 — REPAIR

90.32 %

Auto-applicable rate (n = 279)

Manual: 89.5 % semantic · 68 % combined (n = 25)

R4 — LATENCY

2.2 s

Median stage-1 · p95 = 4.3 s

Comfortably inside on-demand band (≤ 5 s)

R5 — PRIVACY

100 %

leakFreeRate · 12-case prompt-injection harness

schemaValidRate 83 % (10/12) · zero non-loopback traffic

R2 — CONSISTENCY: 100 % JSON-parse success, 3,030 invocations (inter-run agreement is deterministic by seed). R1 / R3 / R4 / R5 detailed on the next four slides.

R1 — Detection Accuracy

qwen3:8b + RAG · canonical taxonomy · full 101-case run

71.43 %

F1

72.46 %

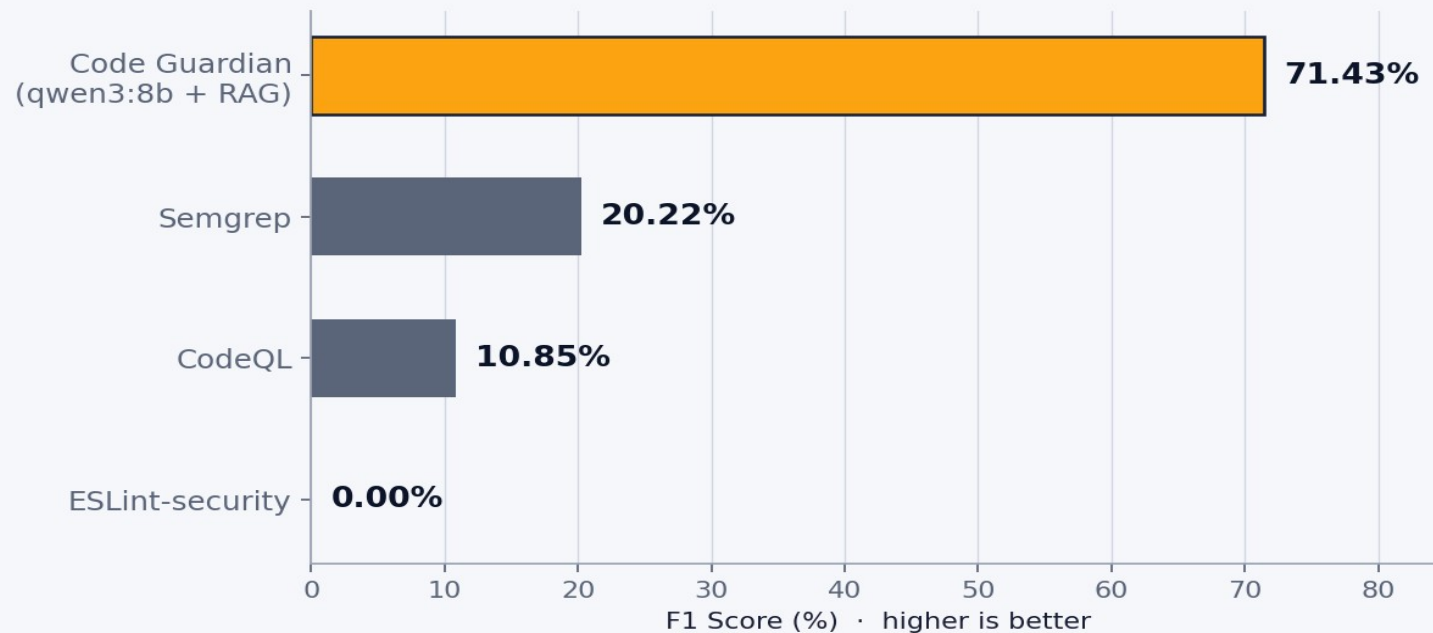
Precision

70.42 %

Recall

10.0 %

FPR



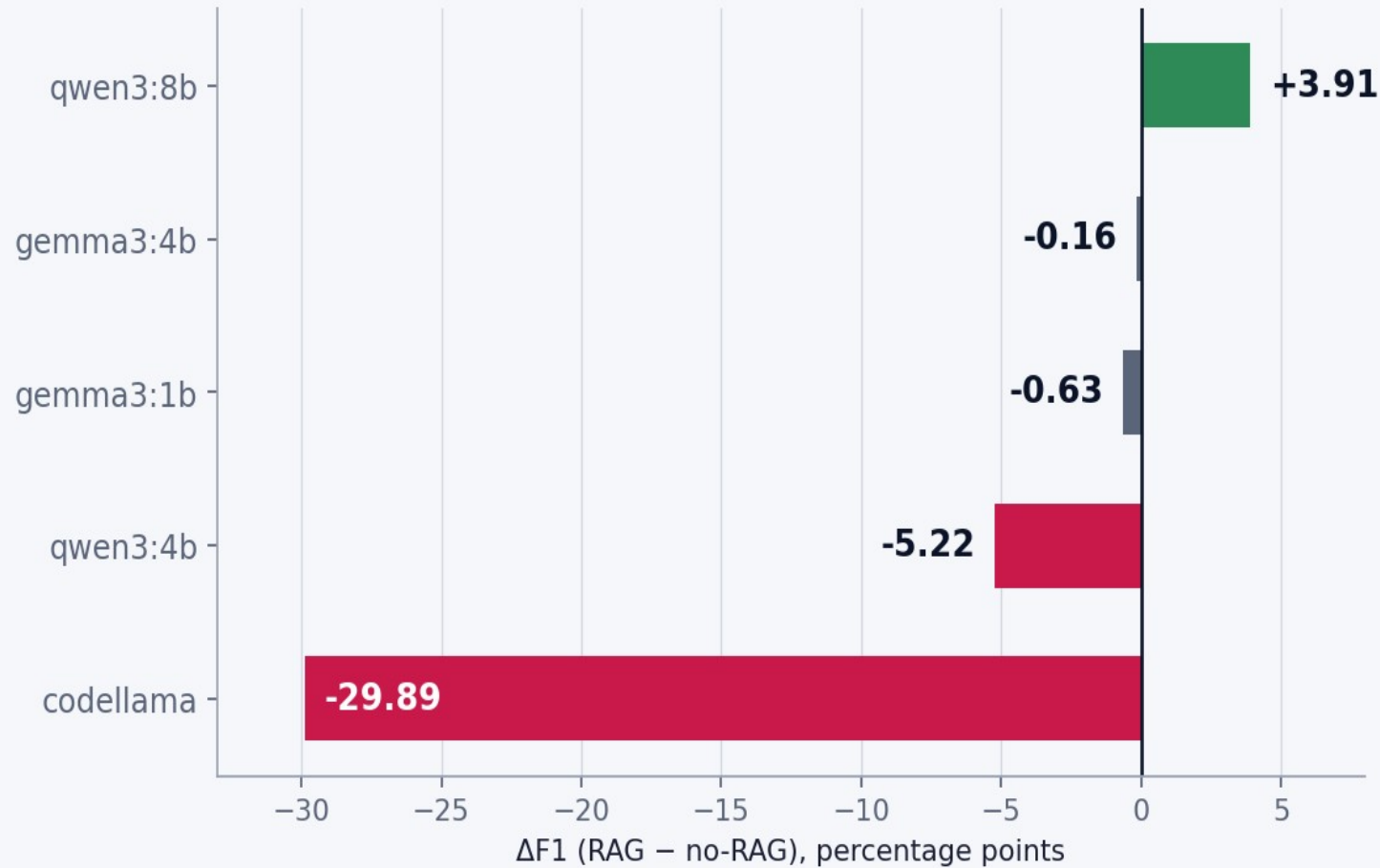
Refinements

With confidence gate ≥ 0.67 :
F1 = 74.07 % · precision = 78.13 %

Held-out 71 / 30 test split:
F1 = 61.11 % · FPR = 0 %

Beats every SAST baseline
on recall-driven F1.

RAG Is Not a Uniform Win



TAKEAWAY

**RAG can help, harm,
or do nothing —
depending on the model.**

Only codellama's -29.89 F1 survives Bonferroni at $\alpha = 0.01$ (McNemar $p = 0.0013$).

Design implication: treat RAG as a per-model configuration choice, not a default.

R3 — Repair Quality

AUTO-APPLICABLE

Validator parses with @babel/parser · fully automated

On issued fixes (252 / 279)

90.32 %

Across all stage-2 calls

84.85 %

MANUAL REVIEW · n = 25

Rubric-based review on a sampled subset

Fix provided

76 %

Semantically correct

89.5 %

Strategy-aligned

94.7 %

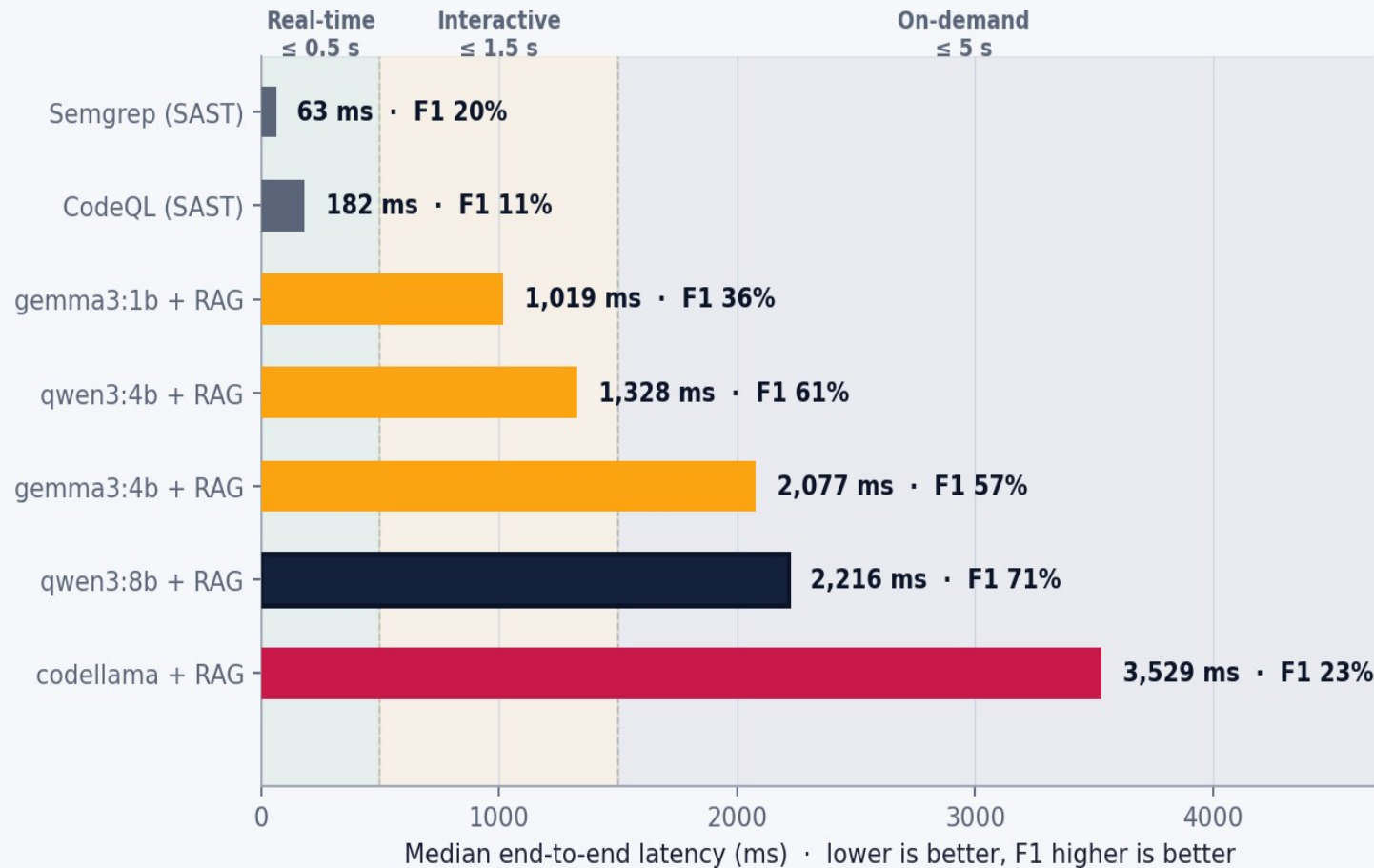
Executable code

42.1 %

TWO-TIER METRIC

Auto-applicable = deployment reading. Manual = decision-support reading. Combined = $76 \% \text{ fix-rate} \times 89.5 \% \text{ semantic} \approx 68 \%$. Stage 2 returns { code, language } + 18 abstentions.

R4 — Usability: Latency Bands



⚡ Real-time ≤ 500 ms

SAST baselines only

🕒 Interactive ≤ 1.5 s

gemma3:1b + RAG (1,019 ms)
qwen3:4b (1,328 ms)

🧠 On-demand ≤ 5 s

qwen3:8b + RAG (2,216 ms) ← headline
p95 = 4,327 ms

R5 — Privacy: Empirically Verified



Network Isolation

Egress harness + tcpdump on the locked config: zero non-loopback connections.

100 % satisfied



Prompt-Injection Harness

12 crafted exfiltration attempts: no code echoed · schemaValidRate 83 % (10/12).

100 % leakFreeRate



Ed25519-Signed Corpus

RAG corpus provenance verified at load — manifest signature checked.

Verified ✓



Reproducible Build

Node 20.19.0-alpine · npm ci · seed = 42
byte-identical runs.

Reproducible ✓

Resource footprint: harness 3 ms median CPU, 87 MB RSS · Ollama 6–8 GB RAM for 8B-q4.

What We Learned



A local 8B model is enough • RQ1

qwen3:8b + RAG reaches ~71 % F1 on JS/TS — well inside the on-demand latency band on a developer laptop. The privacy envelope holds without paying for it in quality.



RAG is model-dependent • RQ2

The largest paired effect in the dataset is a RAG-induced degradation (codellama -29.89 F1). Treat retrieval as a per-model design choice, not a default.



Repair is the real product gap

90 % auto-applicable means the developer can act, not just be warned. This is where SAST falls flat — they detect, they don't fix.



Privacy as a verifiable boundary

Loopback + signed corpus + container pin makes privacy testable, not aspirational. leakFreeRate = 100 % isn't a slogan — it's a measurement.

RQ1 yes — 71.43 % F1 under the privacy envelope • RQ2 yes, but per-model — RAG is not a default • RQ3 yes — 2.2 s median, inside the on-demand band

Code Guardian Closes the Gap

Solution / Approach	R1 ACCURACY	R2 CONSISTENCY	R3 REPAIR	R4 USABILITY	R5 PRIVACY
GitHub Copilot	✓	●	✓	✓	✗
Semgrep	●	✓	✗	✓	✓
CodeQL	●	✓	✗	●	✓
IRIS (GPT-4)	●	●	✗	✗	✗
SecRepair	●	●	●	✗	●
Code Guardian	●	✓	✓	✓	✓

✓ = meets the pre-registered threshold. R1 shown ●. F1 71 % lands in the Medium band — competitive, not High.

LEGEND

✓ High / Satisfied

● Partial

✗ Not satisfied

GAP CLOSED

The only approach to bring privacy + repair + IDE-grade latency together in one local tool — with competitive detection accuracy.

Limitations



Single-reviewer manual review (n = 25)

Inter-rater agreement on the 25-sample subset = future work. Headline R3 metric is the fully objective auto-applicable rate.



Curated 101-case corpus

Selection bias mitigated by held-out 71 / 30 split, 15-case external set (NodeGoat, Juice Shop, 3 CVEs), and whole-project NodeGoat run.



One real-world project end-to-end

NodeGoat · multi-project validation is future work.



Consensus filter inert under deterministic decoding

Three runs are byte-identical with seed = 42. Needs seed rotation to produce a real consistency signal.



No cloud-LLM baseline

Excluded by the privacy threat model · floor provided by 3 SAST baselines.

Future Work



Seed rotation

Genuine consensus signal beyond the deterministic-decoding floor.



Inter-rater agreement

Second reviewer on the 25-sample subset for κ statistic.



Multi-project real-world validation

Beyond NodeGoat · measured recall on diverse production codebases.



Streaming partial results

Break the real-time band by surfacing first-token findings.



Adaptive RAG gating

Per-model RAG enable/disable driven by slide 16's finding.

Contributions of This Thesis



Working VS Code extension

Local LLM + RAG + signed corpus + pinned container — published to the Marketplace.



Two-stage pipeline with structured contracts

Detection (JSON-mode) + Repair ({ code, language }) — schema-pinned at every boundary.



Empirical study

5 models × 2 modes + 3 SAST baselines · 303 invocations per config · CWE-mapped JS/TS corpus.



Reproducibility infrastructure

Byte-identical runs, Ed25519-signed manifest, pinned Node 20.19.0-alpine container.



Statistical discipline

Bonferroni-corrected McNemar, exact-binomial CIs, held-out 71 / 30 split.

References

[1] Touvron, H., et al.: Llama 2: Open foundation and fine-tuned chat models. *arXiv:2307.09288* (2023)

[2] Lewis, P., et al.: Retrieval-augmented generation for knowledge-intensive NLP tasks. *NeurIPS* (2020)

[3] Li, Z., et al.: IRIS: Using LLMs for context-aware vulnerability detection. *ICLR* (2025)

[4] Islam, N., et al.: SecRepair: LLM-driven security vulnerability repair. *NDSS* (2024)

[5] Kavian, S., et al.: LLMSecGuard: Securing LLM-generated code. *EASE* (2024)

[6] Peng, B., et al.: CWEval: Outcome-driven evaluation of LLM code generation. *ACL* (2025)

[7] Pearce, H., et al.: SecurityEval dataset: mining vulnerability examples. *MSR4P&S* (2022)

[8] MITRE: Common Weakness Enumeration (CWE) · <https://cwe.mitre.org>

[9] OWASP Foundation: OWASP Top 10 · <https://owasp.org/Top10>

[10] Malfa, E.L., Petrov, A., et al.: A taxonomy of evaluation methods for LLMs. *arXiv:2402.13887* (2024)

[11] Ollama: Local LLM serving · <https://ollama.com>

[12] Microsoft: VS Code Extension API · <https://code.visualstudio.com/api>

Thank You

Questions & Discussion



HEADLINE

F1 71.43 % · repair 90.32 % auto-applicable · latency 2.2 s median · leakFreeRate 100 %

A1 • Full Per-Model Results • Canonical Taxonomy

Configuration	F1	Prec	Recall	FPR	Latency p50	VRAM
qwen3:8b + RAG	71.43	72.46	70.42	10.00	2,216 ms	~6 GB
qwen3:8b (no RAG)	67.52	61.63	74.65	50.00	1,534 ms	~6 GB
qwen3:4b + RAG	60.92	51.46	74.65	100.00	1,328 ms	~4 GB
qwen3:4b (no RAG)	66.14	56.95	78.87	90.00	1,305 ms	~4 GB
gemma3:4b + RAG	57.14	46.85	73.24	100.00	2,077 ms	~4 GB
gemma3:4b (no RAG)	57.30	46.49	74.65	100.00	2,550 ms	~4 GB
gemma3:1b + RAG	36.02	32.22	40.85	23.33	1,019 ms	~2 GB
gemma3:1b (no RAG)	36.65	29.17	49.30	53.33	1,215 ms	~2 GB
codellama + RAG	23.17	14.79	53.52	100.00	3,529 ms	~6 GB
codellama (no RAG)	53.06	41.60	73.24	100.00	2,024 ms	~6 GB
Semgrep (SAST)	20.22	50.00	12.68	6.67	63 ms	n/a
CodeQL (SAST)	10.85	12.07	9.86	53.33	182 ms	n/a
ESLint-security	0.00	0.00	0.00	0.00	1 ms	n/a

qwen3:8b + RAG headline uses phase1-b3 canonical run; all others use canonical-taxonomy ablation. p95 latencies in source JSON.

A2 • Confusion Matrices • qwen3:8b + RAG

Full corpus (n = 101) • case-level majority

	Predicted: Vuln	Predicted: Safe
Actual Vuln	50 TP	21 FN
Actual Safe	3 FP	27 TN

Held-out test split (n = 30)

	Predicted: Vuln	Predicted: Safe
Actual Vuln	11 TP	10 FN
Actual Safe	0 FP	9 TN

Sample-level via $\geq 2/3$ majority over 3 deterministic runs (seed=42, T=0). Precision is finding-level/pooled, so 72.46 % / $TP/(TP+FP)$ here. Source: phase1-b3 canonical taxonomy.

A3 • Why the Single-Reviewer Manual Review Is Bounded

Fix provided

76 %

Of 25 reviewed samples, 19 included a repair suggestion.

Semantically correct

89.5 %

Fix addresses the CWE (17 of 19 issued fixes).

Strategy-aligned

94.7 %

Right repair pattern (parameterized query, escaping, etc.).

Executable code

42.1 %

Patch runs without modification; the rest give prose guidance.

Inter-rater on the 25-sample subset = future work. Headline R3 is the fully objective auto-applicable rate (90.32 %, n = 279).

A4 • Why a Curated Corpus, Not an Existing Benchmark

WHY EXISTING JS/TS BENCHMARKS DON'T FIT

Juliet (NIST)

C / C++ / Java only. No first-party JS port.

OWASP Benchmark

Java only.

SecurityEval (Pearce 2022)

Small, partially synthetic. CWE labelling inconsistent with OWASP Top 10.

CyberSecEval

29.3 % of vulnerable samples are reproducible (Peng et al. 2025).

CWEval (Peng 2025)

119 curated tasks across 31 CWEs, 5 languages — peer-comparable to 101 cases.

SELECTION-BIAS MITIGATIONS IN THIS THESIS

Held-out 71 / 30 split

Frozen TEST set keeps threshold tuning out of headline numbers.

15-case external corpus

NodeGoat, Juice Shop, 3 named CVEs — independent of the curator.

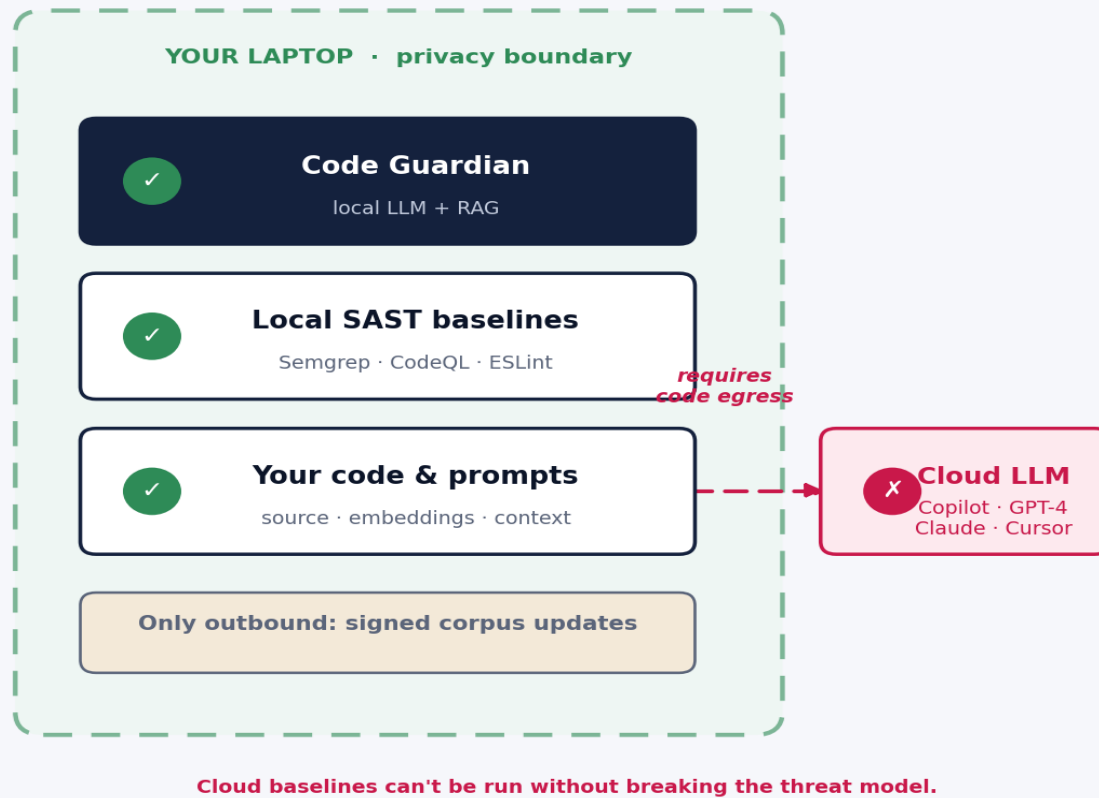
Whole-project NodeGoat run

Project-level recall independent of sample selection.

Provenance bound into manifest

Ed25519 signature; corpus cannot be silently rewritten.

A5 • Why No Cloud-LLM Baseline



ARGUMENT

Cloud-LLM is the problem statement

Including it as a baseline contradicts the threat model.

Floor provided by 3 SAST baselines

Semgrep, CodeQL, ESLint · all local, all reproducible.

Frontier cloud will always win raw F1

The thesis competes on the constraint set: local, zero-egress, reproducible.

Post-defense addendum

A one-paragraph comparison to published GPT-4 numbers on similar JS corpora is straightforward.

A6 • Why R2 Reports 100 % Inter-Run Agreement

RUN 1

```
{
  "findings": [
    {
      "category":
        "sql-injection",
      "severity":
        "high",
      "line": 42
    }
  ]
}
```

RUN 2

```
{
  "findings": [
    {
      "category":
        "sql-injection",
      "severity":
        "high",
      "line": 42
    }
  ]
}
```

RUN 3

```
{
  "findings": [
    {
      "category":
        "sql-injection",
      "severity":
        "high",
      "line": 42
    }
  ]
}
```

WHY THEY'RE IDENTICAL

temperature = 0 • seed = 42 • format = JSON • fixed prompt. Byte-identical → consensus signal requires seed rotation (future work).